Árboles

Definición

Un **árbol** es un grafo conexo que no tiene ciclos.

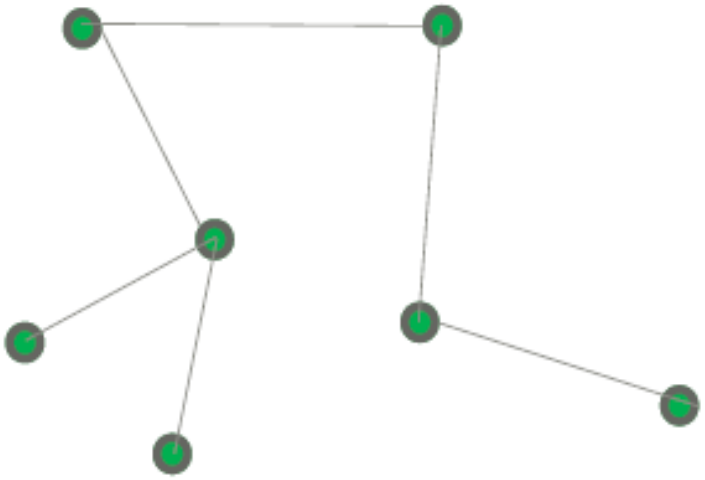


Figura 1: Ejemplo de árbol

Definición

Una **hoja** en un grafo es un vértice de grado 1.

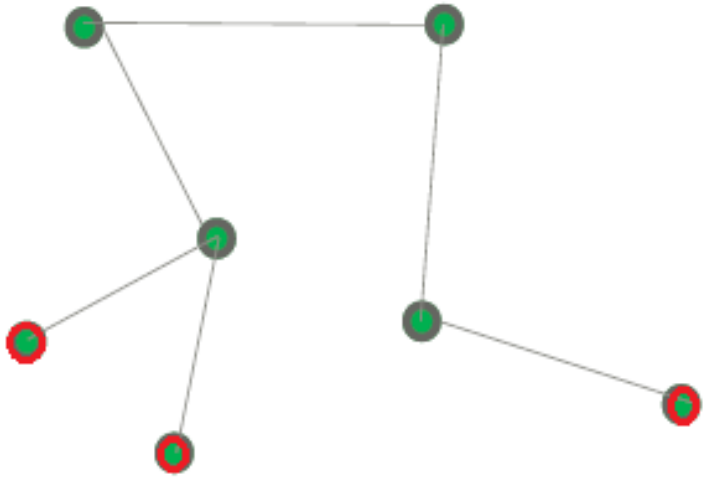


Figura 2: Hojas

Proposición 1 (*)

Todo árbol con por lo menos una arista tiene por lo menos dos vértices de grado 1.

Dem: Tomemos en el árbol un camino simple maximal $< v_1, v_2, ..v_n >$. Veamos que $g(v_1) = g(v_n) = 1$. Si suponemos que no es así, por ejemplo, si suponemos que el $g(v_1) = 2$, además de v_2 tendrá otro adyacente.

Si fuera alguno de los v_i, $3 \leq i \leq n$ tendríamos un ciclo en el grafo lo cual es absurdo pues no sería un árbol. Si fuera adyacente a un vértice w que no esté en el camino se contradice la maximalidad del camino elegido, luego $g(v_1) = 1$. El análisis es análogo para v_n.

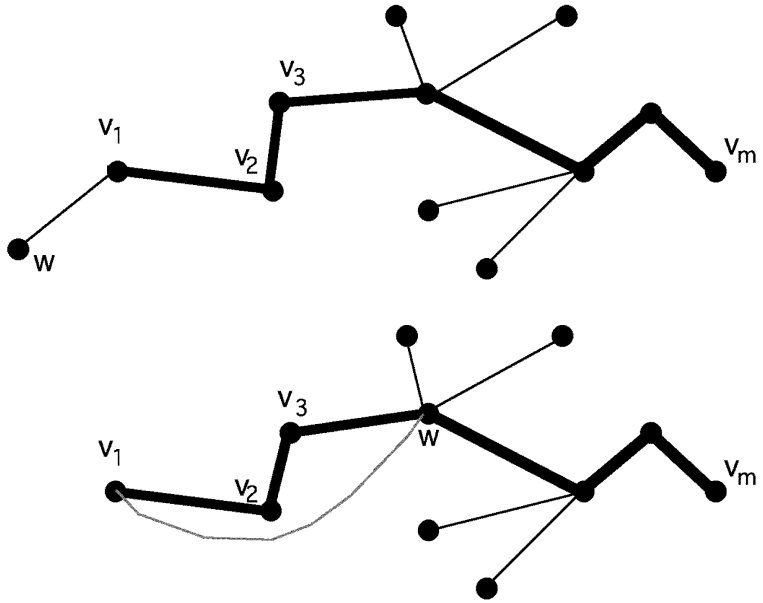


Figura 3: Los dos casos de la proposición anterior.

Corolario 2

Si el grado de todos los vértices de un grafo es mayor o igual a 2 entonces el grafo debe contener al menos un ciclo.

Proposición 3 (*)

Todo árbol con n vértices contiene exactamente $n-1$ aristas.

Dem: Observemos que al no tener ciclos el grafo es plano porque es imposible que tenga un subgrafo homeomorfo a K_5 o a $K_{3,3}$. Además como no tiene ciclos el grafo tiene una sola región: la infinita. Por el teorema de Euler sabemos que

$$v - e + r = 2 \Longrightarrow n - e + 1 = 2 \Longrightarrow e = n - 1.$$

Definición

Un **bosque** es un grafo acíclico, o sea, una unión disjunta de árboles.

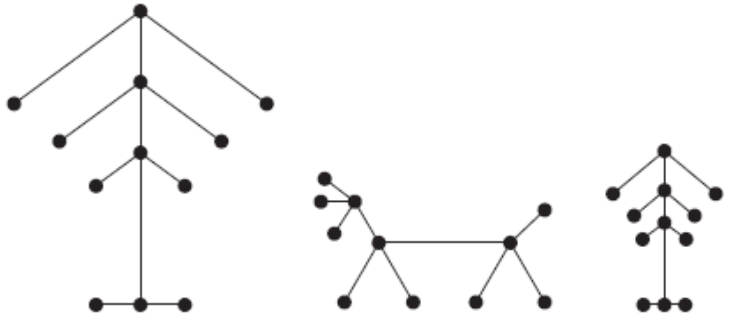


Figura 4: Ejemplo de bosque

Corolario 4

Un bosque F con n vértices tiene $n - \kappa(F)$ aristas.

Nombre: $\kappa(G)$ es el **número de componentes** de un grafo G.

Corolario 5

Un grafo G con n vértices tiene por lo menos $n - \kappa(G)$ aristas.

Proposición 6

Sea un grafo G simple con n vértices y k componentes entonces la cantidad de aristas es

$$\#(E_G) \leq \frac{(n-k)(n-k+1)}{2}$$

<u>Dem</u>: Sean $C_1, C_2, ...C_k$ las k componentes de G. Si la componente C_i tiene n_i vértices entonces $\sum_{i=1}^{k} n_i = n$ y $n_i \geq 1$. Como el máximo número de aristas se obtiene cuando cada una de las componentes es un grafo completo, podemos suponer que $\#(E_{C_i}) = \dfrac{n_i^2 - n_i}{2}$. Así, $\#(E_G) = \sum_{i=1}^{k} \dfrac{n_i^2 - n_i}{2}$.

Para establecer la cota superior, basta ver que el máximo número de aristas se alcanza cuando un grafo tiene exactamente $k-1$ componentes triviales y la restante es un grafo completo $K_{n-(k-1)}$ en este caso la cantidad de aristas es $\dfrac{(n-k)(n-k+1)}{2}$.

Corolario 7

Un grafo simple con n vértices y más de $\dfrac{(n-1)(n-2)}{2}$ aristas debe ser conexo.

<u>Dem</u>: Surge de reemplazar en $\dfrac{(n-k)(n-k+1)}{2}$ el valor de k por 2 que es la máxima cantidad de aristas para un grafo con dos componentes.

Seis diferentes caracterizaciones de un árbol

Teorema (*)

Sea T un grafo con n vértices (sin lazos). Entonces las siguientes afirmaciones son equivalentes:

1. T es un árbol.
2. T no contiene ciclos y tiene $n-1$ aristas.
3. T es conexo y tiene $n-1$ aristas.
4. T es conexo y toda arista es una arista de corte.
5. Todo par de vértices de T están conectados por exactamente un camino simple.
6. T no contiene ciclos y $T+e$ tiene exactamente un ciclo.

<u>Dem</u>: $(1 \Rightarrow 2)$ T es un árbol: por definición no contiene ciclos y ya demostramos por Euler que tiene $n-1$ aristas.

$(2 \Rightarrow 3)$ Que tiene $n-1$ aristas es hipótesis. Supongamos que T tiene k componentes, entonces el bosque tiene $n-k$ aristas ver Corolario 4 entonces $k=1$ entonces T es conexo: es un árbol.
$(3 \Rightarrow 4)$ Que T es conexo es hipótesis. Sea e una arista de T entonces $T-e$ contiene $n-2$ aristas. Por el corolario 5 todo grafo G de n vértices tiene al menos $n-\kappa(G)$ aristas, entonces $T-e$ tiene al menos dos componentes por lo que e es una arista de corte.
$(4 \Rightarrow 5)$ Que todo vértice está conectado por un camino se deduce de la hipótesis de que es conexo. Supongamos que existen dos vértices u y v unidos por dos caminos distintos: dichos caminos pueden tener aristas en común, pero en un momento los caminos divergen supongamos que eso sucede en un vértice x y que vuelven a encontrarse en un vértice y, entonces existe un ciclo que contiene a los vértices x e y entonces hay aristas en el grafo que no son de corte. Absurdo.

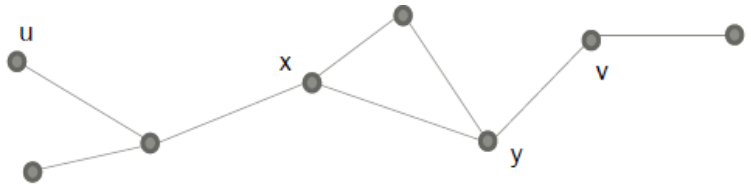


Figura 5: $(4 \Rightarrow 5)$

$(5 \Rightarrow 6)$ T no contiene ciclos ya que por hipótesis todo par de vértices está unido por exactamente un camino simple, si hubiera un ciclo todos sus vértices (los del ciclo) estarían unidos por dos caminos simples contradiciendo la hipótesis. La adición de una arista $e = \{a, b\}$ entre un par cualquiera de vértices a, b genera un ciclo que surge de la concatenación del camino simple que existe por hipótesis con $< a, e, b >$. No puede generar más de un ciclo porque eso significaría que existía un ciclo antes de agregar la arista.
$(6 \Rightarrow 1)$ Que T no contiene ciclos es hipótesis. Faltaría ver que es conexo. Supongamos que no, entonces el grafo tiene al menos dos componentes conexas. Si tomamos un par de vértices en distintas componentes y agregamos una arista entre ellos esta no formaría un ciclo contradiciendo la hipótesis. Así queda probado que los 6 enunciados son equivalentes.

Árboles como subgrafos

Un grafo puede contener un número de diferentes árboles como subgrafos. El siguiente teorema muestra que si un grafo es *suficientemente denso*, entonces contiene a T como subgrafo.

Teorema (*)

Sea T un árbol con n vértices y sea G un grafo simple tal que $\delta_{min}(G) \geq n-1$ entonces T es un subgrafo de G.

<u>Dem</u>: Si $n = 1$ o $n = 2$ es obviamente cierto ya que K_1 y K_2 son subgrafos de todo grafo con al menos una arista.
Hi) Supongamos que vale para cierto $n \geq 2$.
Ti) Veamos que vale para $n+1$.
Sea T un árbol con $n+1$ vértices y sea G un grafo cuyo $\delta_{min}(G) \geq (n+1)-1 = n$. Tomemos un vértice v que sea una hoja del árbol y sea u el único adyacente a v en el árbol.

Vemos que $T - v$ es un árbol con n vértices. Por Hi), $T - v$ es subgrafo de G ya que $\delta_{min}(G) \geq n \geq n - 1$. Como $\delta_{min}(G) \geq n$, existe en G un vértice w que no está en $T - v$ y que es adyacente a u. Sea $e = \{u, w\}$ entonces $T - v$ junto con la arista e y el vértice w forman un grafo que es isomorfo a T y es subgrafo de G.

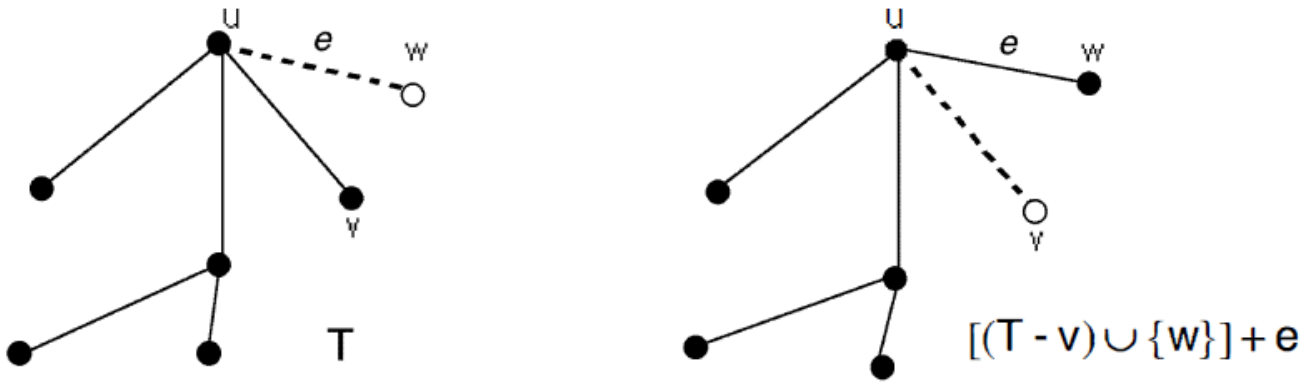


Figura 6: T es isomorfo a $[(T - v) \cup \{w\}] + e$

Árboles con raíz, árboles ordenados y árboles binarios

Definición

Un **árbol dirigido** es un digrafo cuyo grafo subyacente es un árbol.

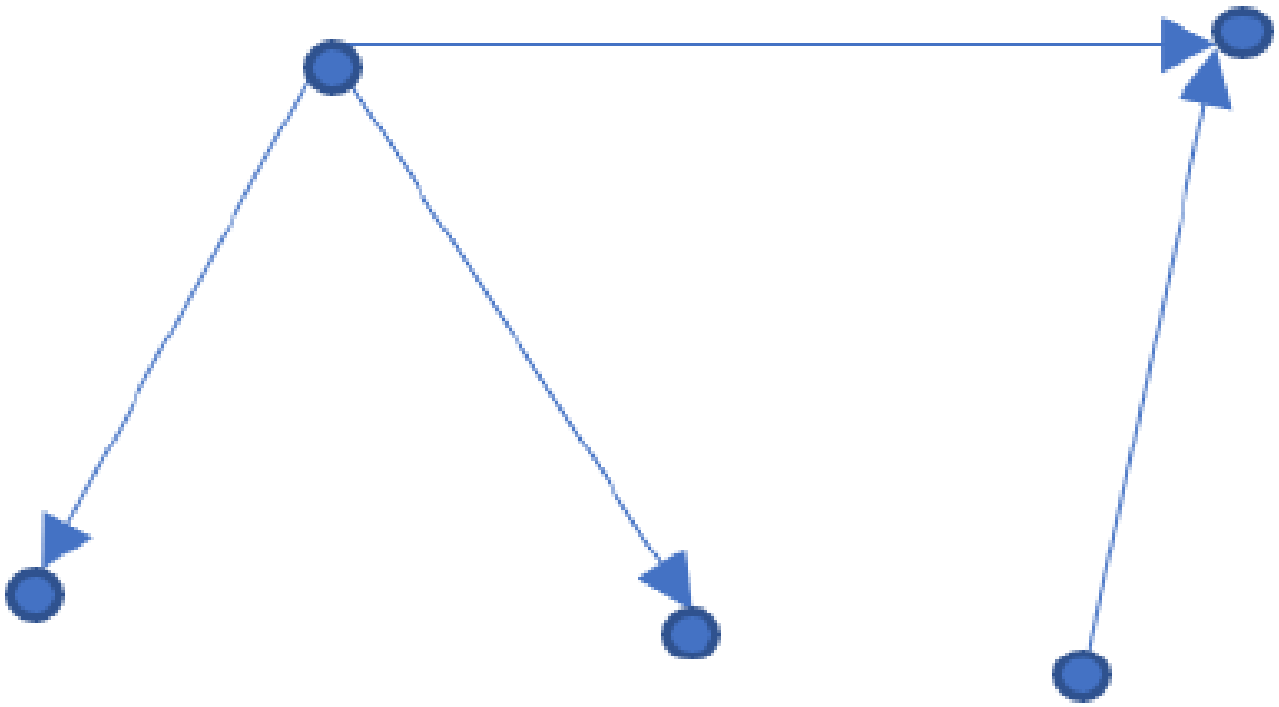


Figura 7: Ejemplo de árbol dirigido

Definición

Un **árbol con raíz** es un árbol dirigido con un vértice distinguido r, llamado raíz, tal que para todo otro vértice v hay un camino directo de r a v.

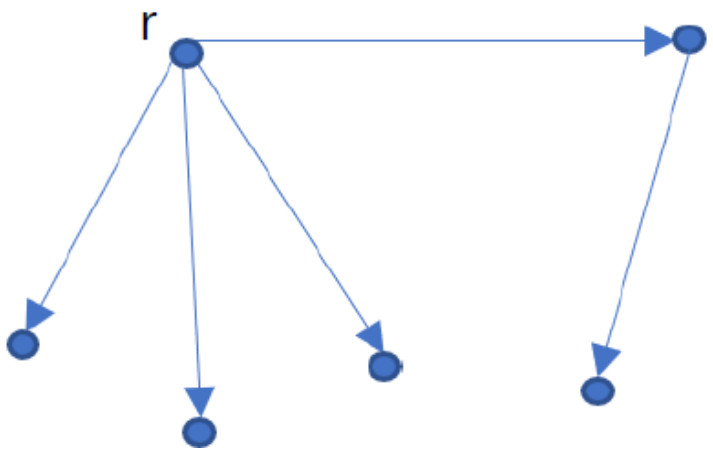


Figura 8: Ejemplo de árbol con raíz

Obs:

Un árbol con raíz, es un árbol dirigido, tal que la raíz tiene grado de entrada 0 y todos los demás vértices tienen grado de entrada 1. Al dibujar un árbol con raíz, las flechas usualmente se omiten, pues su dirección siempre es alejándose de la raíz.

Designar una raíz impone una jerarquía sobre los vértices de un árbol con raíz, según a su distancia de esa raíz.

Definición

En un árbol con raíz, la **profundidad** o **nivel** del vértice v es su distancia a la raíz.

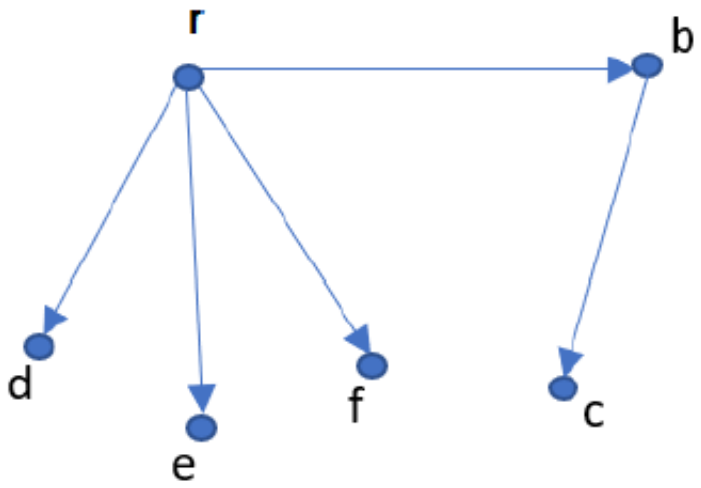


Figura 9: La profundidad o nivel del vértice r es cero pues es la raíz. La profundidad de los vértices d, e, f y b es uno, y la del vértice c es dos.

Definición

La **altura** de un árbol con raíz es la longitud del camino más largo desde la raíz.

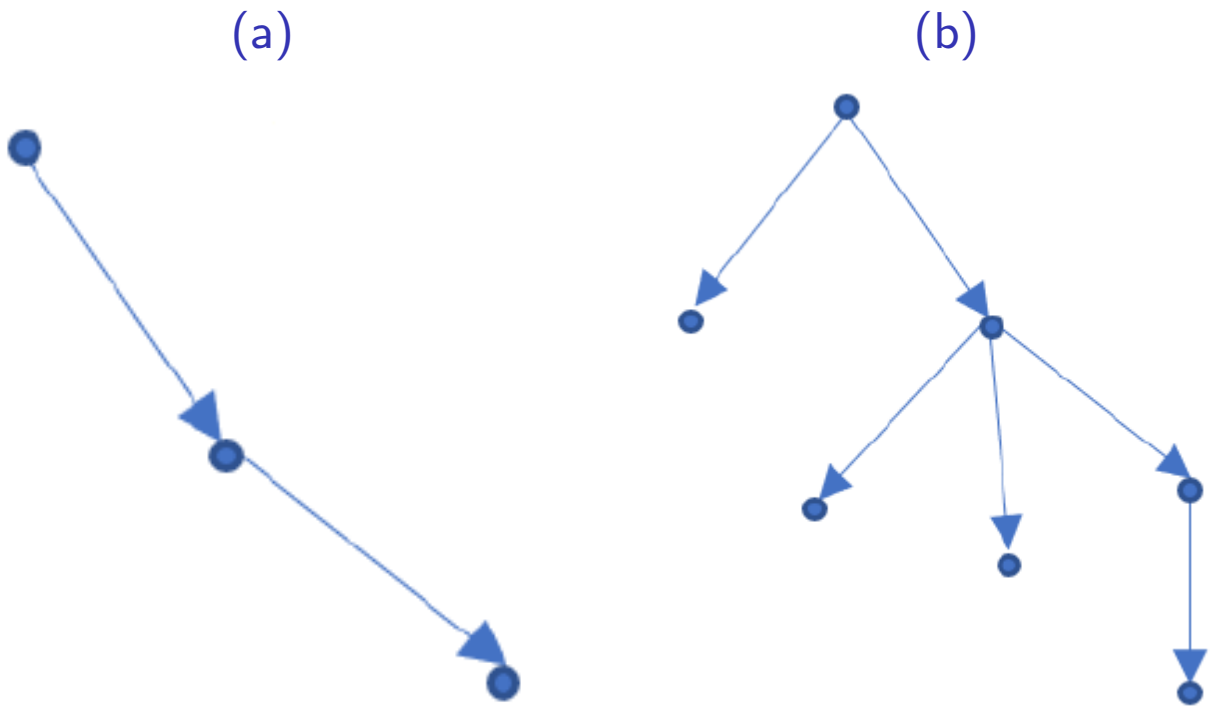


Figura 10: (a) Árbol de altura 2 (b) Árbol de altura 3

Definición

Si el vértice v es el que precede inmediatamente a w en el camino directo $r - w$ entonces v es el **padre** de w y w es el **hijo** de v. Si dos vértices tienen el mismo padre, se llaman **hermanos**.

Definición

Un vértice w es llamado **descendiente** de un vértice v, y v es llamado **antecesor** de w, si v está en el único camino $r - w$. Si $w \neq v$, w es un **descendiente propio** de v, y v un **antecesor propio** de w.

Definición

Un vértice que no tiene hijos se llama **hoja** y un **vértice interno** es un vértice que no es hoja.

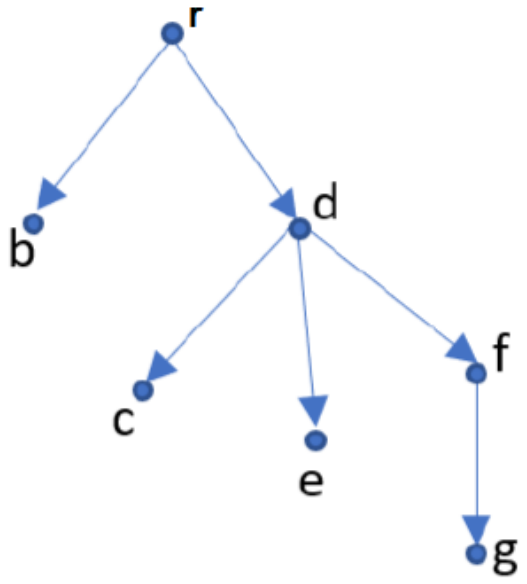


Figura 11

El vértice r es padre de b y d y ellos son sus hijos. El vértice d tiene tres hijos: c, e y f, estos tres últimos son hermanos entre sí.
Todos los vértices son descendientes de r, que es la raíz. El vértice b no es descendiente de d. Los vértices c, e, f y g son descendientes de d. *Todo vértice es descendiente y antecesor de sí mismo.* El vértice g es descendiente propio del vértice d ya que $g \neq d$. Los vértices b, c, e y g son hojas pues no tienen hijos.

Muchas aplicaciones imponen un límite superior al número de hijos que un determinado vértice puede tener.

Definición

Un árbol m-**ario** con $m \geq 2$ es un árbol con raíz en el cual todo vértice tiene m o menos de m hijos.

Definición

Un árbol m-**ario completo** es un árbol m-ario en el cual todos los vértices internos tienen exactamente m hijos y todas las hojas tienen la misma profundidad.

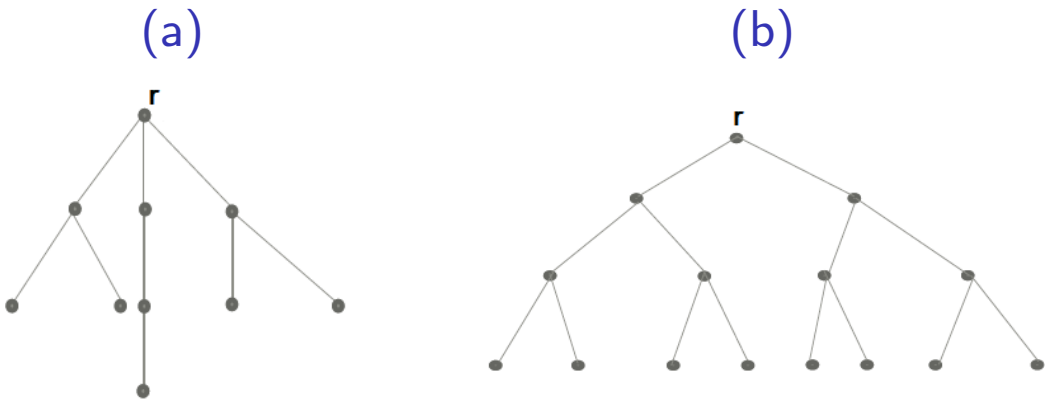


Figura 12: (a) Árbol 3-ario, no completo (b) Árbol binario completo de altura 3

Proposición

Un árbol m-ario tiene a lo sumo m^k vértices en el nivel k.

Dem: Por inducción en k. En el nivel cero sólo tenemos a la raíz: $m^0 = 1$.
HI) Supongamos que en nivel k tenemos a lo sumo m^k vértices.
TI) En nivel $k+1$ tenemos m^{k+1} vértices.
Cada vértice del nivel k tiene a lo sumo m hijos entonces en el nivel $k+1$, tendremos a los sumo $m^k.m = m^{k+1}$ vértices.

Teorema

Sea T un árbol m-ario con n vértices y altura h. Entonces

$$h + 1 \leq n \leq \frac{m^{h+1} - 1}{m - 1}$$

Dem: Si llamamos n_k al número de vértices en el nivel k, por la proposición anterior, $1 \leq n_k \leq m^k$. Así:

$$h + 1 \leq \sum_{k=0}^{h} 1 \leq \sum_{k=0}^{h} n_k \leq \sum_{k=0}^{h} m^k = \frac{m^{h+1} - 1}{m - 1}.$$

Isomorfismo como árboles con raíz

El resultado se sigue porque $\sum_{k=0}^{h} n_k = n$.

Definición

Dos árboles con raíz son **isomorfos** si hay un isomorfismo de grafos entre ellos que mapea raíz en raíz.

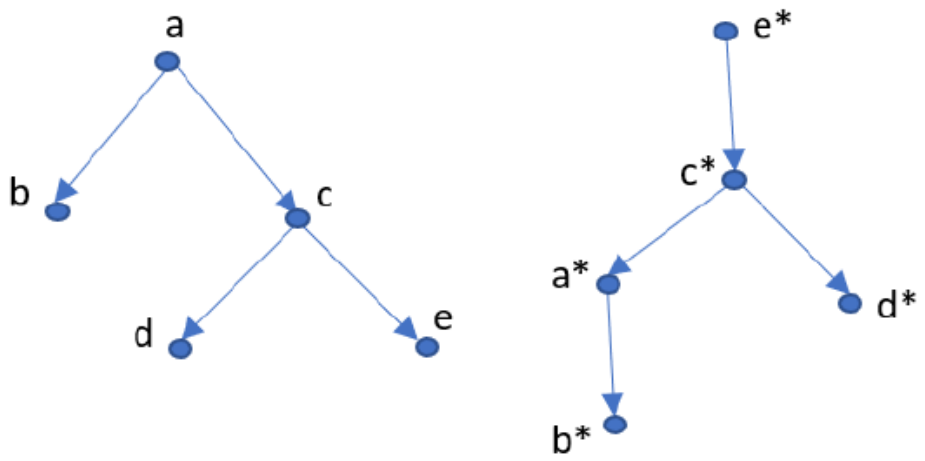


Figura 13: Los grafos subyacentes de estos árboles son isomorfos. Sin embargo no son isomorfos como árboles con raíz.

Árbol ordenado

Definición

Un **árbol ordenado** es un árbol con raíz en el cual cada hijo de cada vértice tiene asignado un orden fijo.

Definición

En un **dibujo plano estándar** de un árbol ordenado, la raíz está arriba de todo, los vértices que están en un mismo nivel horizontalmemte alineados y el orden izquierda-derecha de los vértices concuerdan con su orden prescripto.

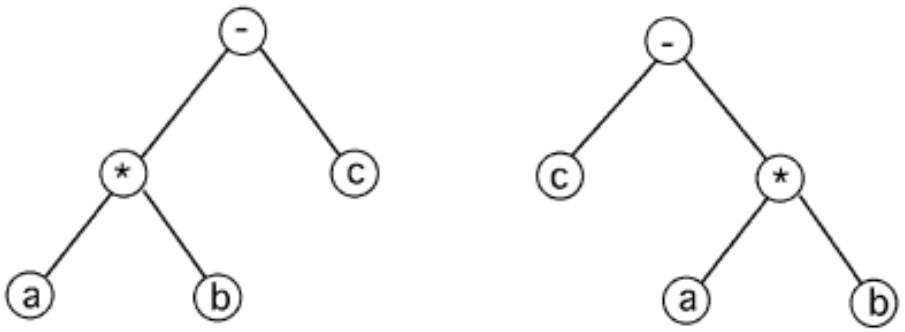


Figura 14: Ejemplos de árboles ordenados. El de la izquierda almacena la expresión $a * b - c$ mientras que el de la derecha almacena $c - a * b$

Árboles binarios

Definición

Un **árbol binario** es un árbol 2-ario en el cual cada hijo es designado o hijo izquierdo o hijo derecho

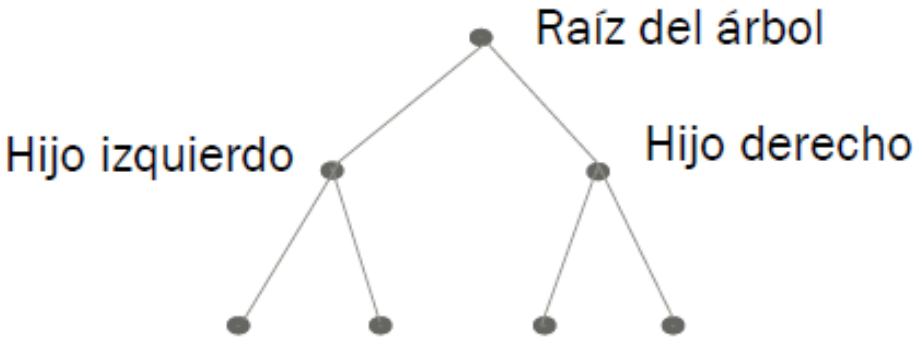


Figura 15: Ejemplo de árbol binario

Definición

El **subárbol izquierdo** (**derecho**) de un vértice v en un árbol binario T es el subárbol binario recubridor del hijo izquierdo (derecho) y todos sus descendientes.

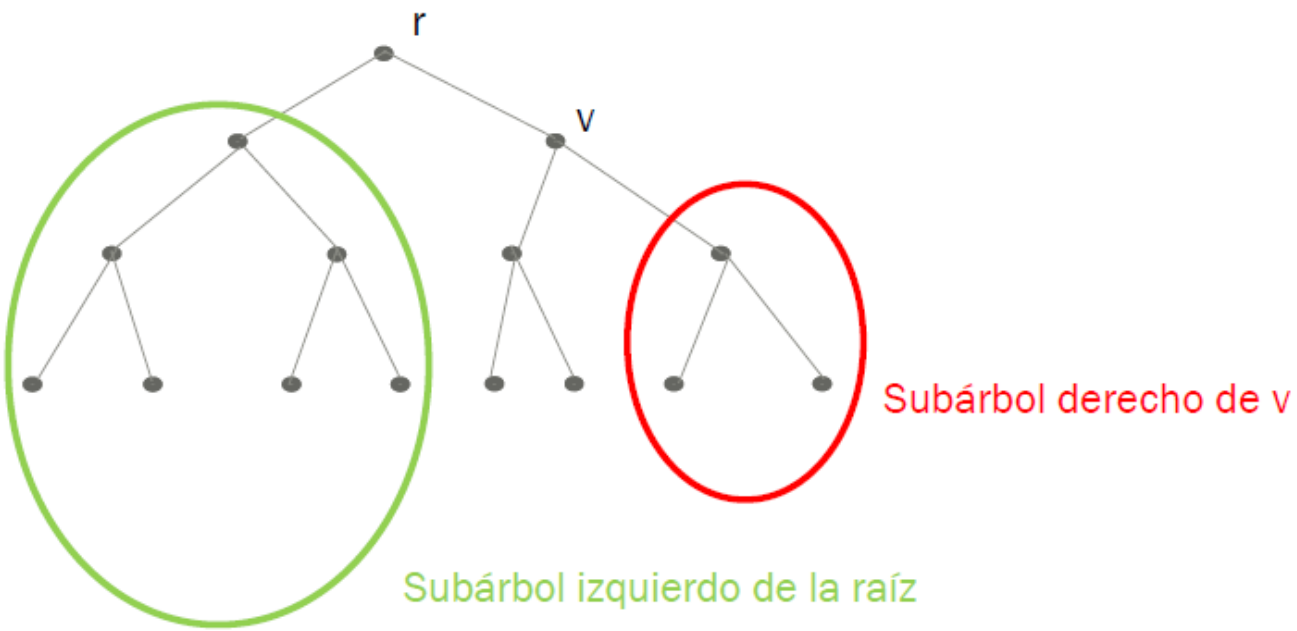


Figura 16

La designación obligatoria de hijo izquierdo o hijo derecho significa que dos árboles binarios pueden ser indistinguibles cuando se los considera árboles ordenados.

Ejemplo:

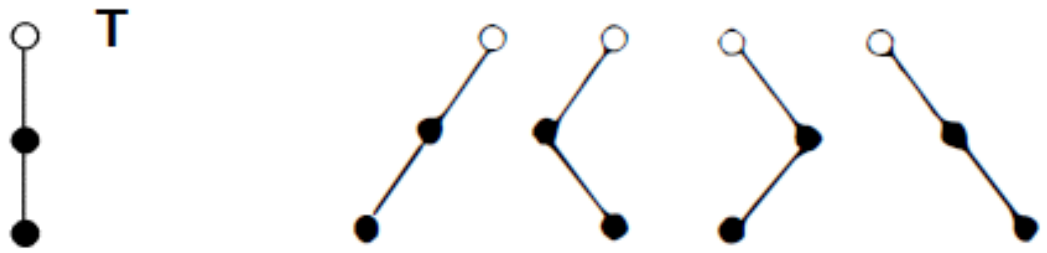


Figura 17: Cuatro árboles binarios distintos, que son el mismo árbol ordenado T.

Teorema (*)

Un árbol binario completo de altura h tiene $2^{h+1} - 1$ vértices.

Dem: Si $h = 0$, la afirmación es válida. Supongamos que $k \geq 0$ y que un árbol binario completo de altura k tiene $2^{k+1} - 1$ vértices. Sea T un árbol binario de altura $k + 1$. Como T es completo, sus árboles izquierdo y derecho, digamos T_1 y T_2, también deben ser completos. Además, T_1 y T_2 tienen ambos altura k, entonces por hipótesis inductiva, tienen $2^{k+1} - 1$ vértices.

Así, el número total de vértices de T es:

$$1 + 2^{k+1} - 1 + 2^{k+1} - 1 = 2^{k+2} - 1.$$

Corolario

Todo árbol binario de altura h tiene a lo sumo $2^{h+1} - 1$ vértices.

Codificación de Huffman

Un alfabeto $A = \{\alpha, \beta, ...\}$ es un conjunto finito de símbolos que llamamos **letras**. Una sucesión de letras la llamamos **palabra**. Una sucesión de palabras la llamamos **mensaje**. Queremos transmitir un mensaje usando sólo ceros y unos. Para ello representaremos a cada letra del alfabeto por una sucesión de ceros y unos que llamamos **código de la letra**. Dos letras distintas deben tener diferente código. Formalmente:

Definición

Un **código binario** es una asignación de símbolos a cadenas de bits. Cada cadena de bits se llama **palabra** del código.

Además, ningún código debe ser *prefijo* de otro código.

Definición

Un **código prefijo** es un código binario con la propiedad que ninguna palabra del código es una subpalabra inicial de otra palabra de código.

Por ejemplo, si el código de $\alpha \in A$ fuera 101 y el código de β fuera 10 entonces en el mensaje 101... no sabríamos si empieza con α o con β.

Podríamos usar un espacio entre letra y letra pero entonces la codificación tendría tres elementos: **0,1 y "espacio "**.
Otro ejemplo:
$000 \to a$, $001 \to b$, $01 \to c$, $11 \to d$, $10 \to e$ Es un código prefijo.
Porque ninguna codificación de los símbolos es subpalabra de otra. En este caso los símbolos pueden, en general, tener distinta longitud, entonces se elige la longitud más corta para los símbolos que más se repiten.

Ejemplo:

Supongamos que un mensaje puede expresarse por palabras del alfabeto $\{a, b, c, d, e, f, g\}$. En el árbol binario, colocamos los símbolos del alfabeto que se quieren codificar. Luego, se etiquetan las aristas que van a un hijo izquierdo con 0 y a las que van a un hijo derecho con 1.

Letra	a	b	c	d	e	f	g
Código	000	0010	0011	0101	011	100	101

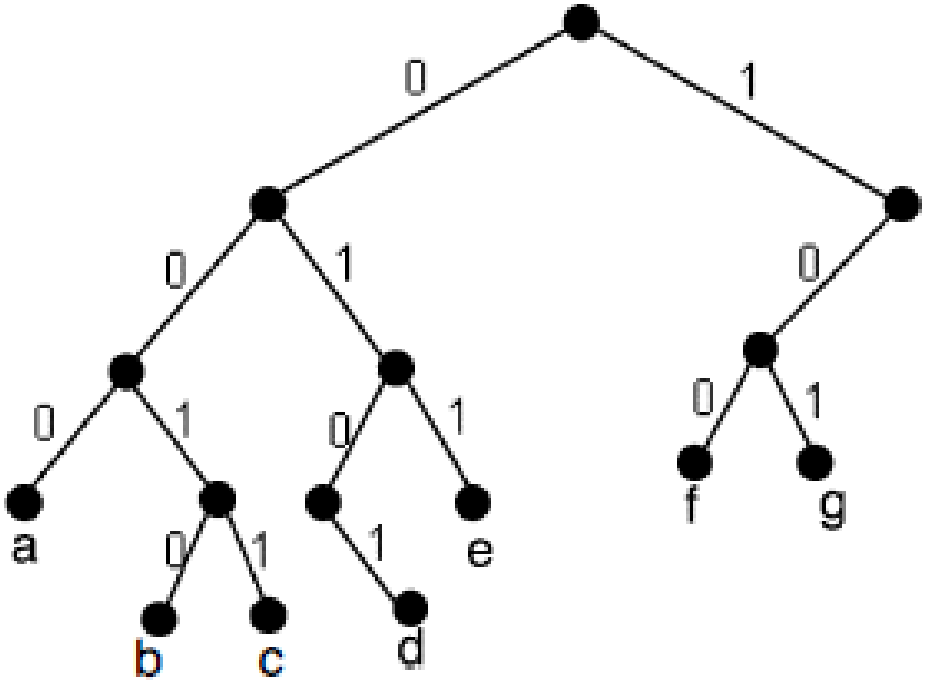


Figura 18: Un árbol binario usado para construir un código prefijo.

El código que armamos... ?´es un "buen´´ código?

Definición

Una medida de eficiencia de un código es el **promedio ponderado** del largo de las palabras de código, donde el largo de cada palabra de código (# bits de la cadena) es multiplicada por la frecuencia del símbolo que representa (# apariciones del símbolo / total).

Ejemplo: supongamos que aquello que queremos codificar tiene las siguientes frecuencias para cada símbolo:

Letra	a	b	c	d	e	f	g
Frecuencia	0.2	0.05	0.1	0.1	0.25	0.15	0.15
Código	000	0010	0011	0101	011	100	101

Calculamos la eficiencia en el código prefijo que construimos anteriormente:

$$3 * 0{,}2 + 4 * 0{,}05 + 4 * 0{,}1 + 4 * 0{,}1 + 3 * 0{,}25 + 3 * 0{,}15 + 3 * 0{,}15 = \mathbf{3.25}$$

Lo ideal es que este número sea lo más pequeño posible.

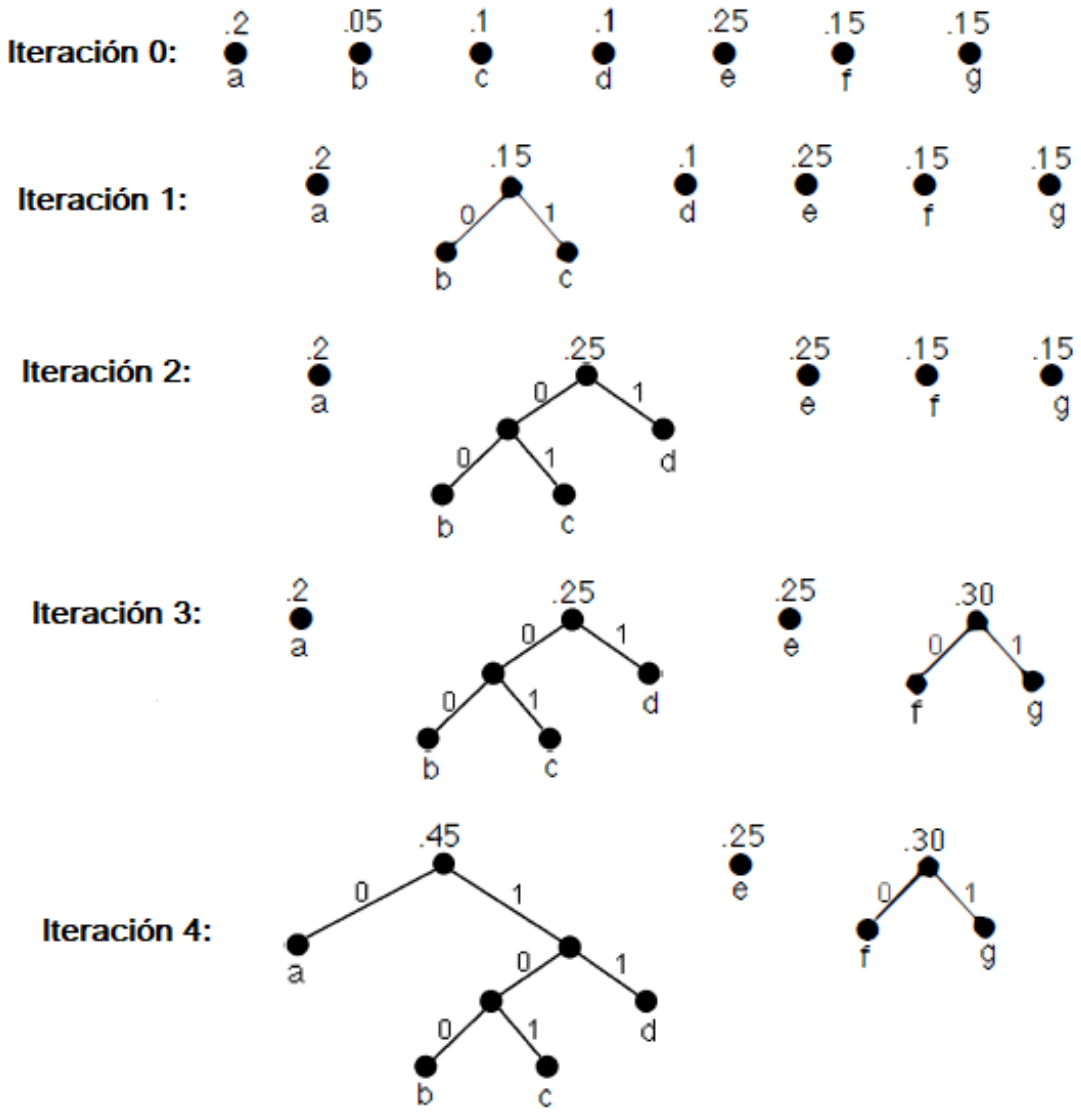
Algoritmo de Huffman de código prefijo

Entrada: $S = \{s_1, \cdots, s_m\}$, $W = \{w_1, \cdots, w_m\}$.
Salida: Un árbol binario F representante del código prefijo de S donde el largo de las palabras de código tienen el mínimo promedio ponderado.

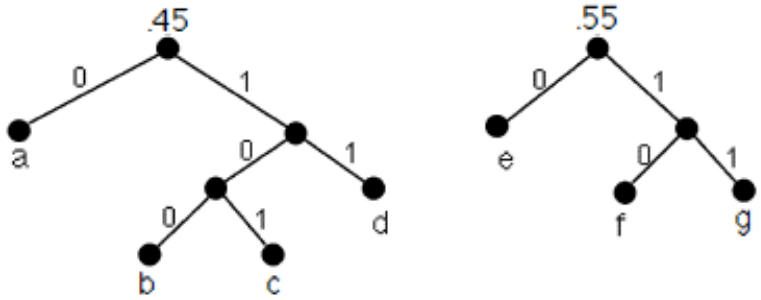
1. FOR I:=1 TO m
2. {
3. Elegir de F dos árboles T_1 y T_2 de peso mínimo.
4. Crear un nuevo árbol cuyo subárbol izquierdo sea T_1 y cuyo subárbol derecho sea T_2.
5. Etiquetar la arista que va a T_1 con 0 y la que va a T_2 con 1.
6. Asignar al nuevo árbol peso $w(T_1) + w(T_2)$.
7. Reemplazar T_1 y T_2 por el nuevo árbol en F.
8. }
9. RETURN F.

Figura 19: Los empates se resuelven según el orden izquierda-derecha de los árboles.

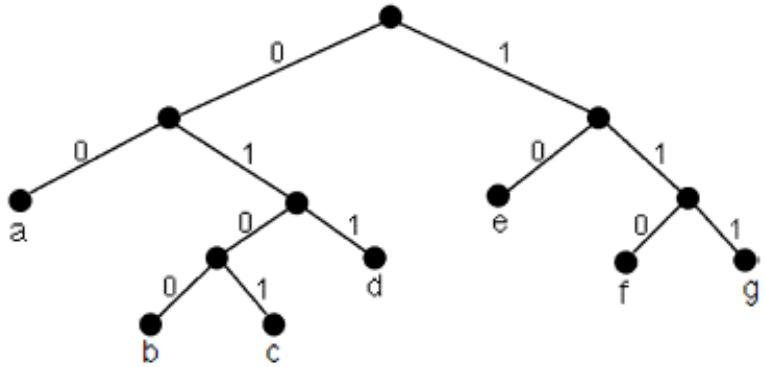
Ejemplo: Volvamos a los símbolos/frecuencias que teníamos anteriormente y apliquemos el algoritmo de Huffman para encontrar un código prefijo eficiente.



Iteración 5:



Iteración Final:



Letra	a	b	c	d	e	f	g
Frecuencia	0.2	0.05	0.1	0.1	0.25	0.15	0.15
Código	00	0100	0101	011	10	110	111

Calculemos la eficiencia:

$$2 * 0{,}2 + 4 * 0{,}05 + 4 * 0{,}1 + 3 * 0{,}1 + 2 * 0{,}25 + 3 * 0{,}15 + 3 * 0{,}15 = \mathbf{2.7}$$